

시장 이벤트 판단을 TradingSTM으로 모은 이유와 실행 흐름

작성일: 2026-09-17. 현재 작업 폴더에 적용된 코드와 같은 날의 검증 결과를 기준으로 작성했습니다.

이 문서는 [두 STM과 Region 실행 구조 설명](#)의 후속 설명입니다. 기존 문서가 두 STM, 세 Region, Controller와 Action의 전체 구조를 설명한다면, 여기서는 **시장 접촉 정책이 Controller와 STM에 나뉘어 있던 문제를 어떻게 줄였는지에** 집중합니다.

설명 순서는 다음 네 가지입니다.

- 1. 이전의 문제점
- 2. 수정 사항
- 3. 수정으로 인한 장점과 남아 있는 한계
- 4. 수정 후 코드를 실행 순서대로 따라가는 예시

문서의 “현재 코드”는 설치된 앱의 바이너리가 아니라 작업 폴더의 소스입니다. 이번 작업에서 앱 설치본을 교체하거나 실행을 재시작하지 않았습니다. 예시는 가짜 거래소를 사용하는 테스트와 소스 검토에 근거합니다.

1. 이전의 문제점

1.1 동작 변경과 구조 변경을 먼저 구분해야 합니다

이번 과정을 한 번의 수정으로 설명하면 무엇이 문제였고 무엇을 개선했는지 혼동하기 쉽습니다. 실제로는 다음 세 상태를 거쳤습니다.

단계	상단 접촉 때의 동작	접촉 정책을 판단한 위치
처음 문제를 조사했을 때	G-07이 안전 종료 정책을 적용했습니다. 무포지션·무주문이면 세션을 종료했고, 보유·주문 처리 중에는 청산·조정 경로를 사용했습니다.	Controller의 이벤트 분류와 STM의 전역 전이
상단 동작을 먼저 수정한 뒤	노출이 없으면 하단 감시로 복귀하고, 노출이 있으면 기존 Case 관리를 계속하도록 바꿨습니다.	여전히 Controller의 이벤트 분류와 STM 전이 두 곳
이번 리팩터링 이후	바로 앞 단계의 매매 동작을 유지합니다.	접촉 조건과 후속 동작을 STM의 전역 전이 구현에 모았습니다.

따라서 자동매매가 상단에서 종료되지 않게 만든 것은 **앞선 동작 수정**이고, 같은 정책을 여러 곳에서 고쳐야 했던 구조를 정리한 것이 이번 수정입니다. 이번 리팩터링에서 Case B/C의 매수·매도 기준을 새로 바꾸지는 않았습니다.

1.2 Controller의 이벤트 이름 선택이 사실상 전략 판단이었습니다

기존 Controller에는 `_select_market_event_type()`이 있었습니다. 이름만 보면 시장 데이터의 형식을 바꾸는 함수처럼 보이지만, 실제로는 다음 정보를 읽었습니다.

- 현재가가 상단·하단 밴드에 닿았는가
- 실제 포지션이 열려 있는가
- 미결 주문 ID 또는 아직 제출하지 않은 주문 의도가 있는가

- 현재 하단 이벤트가 존재하는가
- 터치 당시의 30분봉과 지금의 30분봉이 다른가
- Case C가 해당 하단 이벤트의 진입 기회를 소비했는가, 이후 회복했는가

그리고 이 조건에 따라 **UPPER_BAND_TOUCHED**, **LOWER_BAND_TOUCHED**, **NEW_30M_LOWER_BAND_TOUCHED**, **MARKET_DATA_UPDATED** 중 하나를 골랐습니다.

이 선택은 단순한 이름 변경으로 끝나지 않았습니다. TradingSTM의 Region용 해석 함수는 일반 **MARKET_DATA_UPDATED**를 받아야 현재 상태에 맞는 조건 검사 이벤트로 바꿉니다. 예를 들어 B 보유 상태에서는 **RETRY_CASE_B_CONDITION_CHECK**, C 보유 상태에서는 **RETRY_CASE_C_CONDITION_CHECK**로 해석합니다.

즉 Controller가 어떤 이름을 고르느냐에 따라 **STM의 어느 판단 코드까지 도달할 수 있는지가 달라졌습니다.**

이전 구조

시장 평가

- Controller: 포지션·주문·하단 이벤트를 보고 전략 이벤트 선택
- STM: 선택된 이벤트와 현재 상태를 보고 전이 조건 다시 확인
- Controller: 반환된 Action 실행

예를 들어 상단 전이만 “아무 Action도 반환하지 않는다”로 바꾸더라도, 상단 입력이 계속 **UPPER_BAND_TOUCHED**로 전달되면 그 microstep에서 일반 시장 갱신에 대한 B/C 검사 경로가 열리지 않을 수 있습니다. 세션을 종료하지 않는다는 조건만 만족하고, 보유 포지션의 기존 관리까지 계속한다는 조건은 놓칠 수 있는 구조였습니다.

앞선 수정 때 Controller에 “포지션·주문 처리 중이면 상단에서도 **MARKET_DATA_UPDATED**를 전달한다”는 예외가 필요했던 이유가 이것입니다. 당시 STM만 고치면 된다고 예상하기 어려웠던 지점도 바로 이 연결입니다.

1.3 같은 정책의 조건을 여러 위치에서 함께 확인해야 했습니다

새 30분봉의 하단 접촉을 예로 들면, Controller는 소유자 없음·pending 주문 없음·C 소비와 회복·봉 식별자를 보고 이벤트를 분류했습니다. STM의 G-03도 비슷한 조건을 확인했습니다. 상단에서도 Controller의 우회 조건과 G-07의 노출 보호 조건을 함께 읽어야 했습니다.

확인 위치	당시 확인해야 했던 내용	유지보수상의 부담
Controller의 분류 함수	이번 입력을 상단·최초 하단·새 봉 하단·일반 갱신 중 무엇으로 보낼지	새 정책이 STM까지 전달되는지 확인해야 함
G-02/G-03/G-07	현재 상태에서 실제 전이와 Action을 만들 수 있는지	분류 조건과 다른 판단을 하지 않는지 확인해야 함
상단 정책 설정 필드	레지스트리의 상단 정책 이름	설정을 바꿨다고 실행이 바뀌는 것은 아니었음
테스트와 문서	분류와 전이의 기대 동작	서로 다른 위치의 설명·검사를 함께 맞춰야 함

특히 **UpperBandPolicy**와 **upper_band_policy**는 실제 동작을 선택하는 실행 분기에 사용되지 않았습니다. 정책을 설명하는 값이었는데, 이름과 위치 때문에 동작을 바꾸는 설정처럼 읽힐 여지가 있었습니다. 정책 이름과 실제 코드가 어긋나도 이 필드만으로 실행을 통제하지 못했습니다.

문서와 테스트가 따로 존재하는 것 자체는 문제가 아닙니다. 문제는 **실행 판단에 필요한 조건이 여러 소유자에게 나뉘어 있고, 실행에 쓰이지 않는 설정까지 같은 정책을 표현하고 있었다는 점**입니다.

1.4 큐 처리 시점의 재평가는 필요한 동작이었습니다

이전 Controller는 관측을 큐에 넣을 때와 큐에서 꺼낼 때 분류 함수를 호출했습니다. 두 번 호출했다는 사실만으로 불필요한 작업이었다고 볼 수는 없습니다.

관측을 적재한 뒤 처리하기 전까지 앞선 이벤트가 매수 주문을 만들거나 체결을 반영할 수 있습니다. 상단 가격을 처음 관측했을 때 무포지션이었더라도, 실제 판단할 때는 이미 C 포지션을 보유할 수 있습니다. 이때 과거의 무포지션 판단대로 하단 감시로 초기화하면 진행 중인 관리를 훼손합니다.

이번 수정에서는 이 필요성을 유지했습니다. **시장 가격·지표는 해당 관측의 원본을 쓰고, 주문·포지션·전략 진행 상태는 큐에서 꺼내 판단하는 시점의 값을 씁니다.** 전략 판단 위치를 STM으로 옮기면서 처리 시점의 최신 실행 상태를 반영하도록 했습니다.

2. 수정 사항

2.1 현재 책임 분담

MarketDataController: 가격·봉에서 시장 평가 준비
 → TradingController: 입력 검증, 원본 관측을 큐에 보존
 → event processor: 관측을 꺼내 Controller에 Context 준비 요청
 → TradingSTM: 전역 접촉 정책 → 필요한 경우 Region별 판단
 → TradingSTMResult: 다음 상태와 순서가 있는 Action 요청
 → TradingController: Context·주문·timer에 실제 적용
 → 결과 기록·지표 게시·후속 이벤트 처리

구성 요소	현재 책임	이번 변경
TradingController	입력과 version 검증, 큐 등록, 경과 시간·scope 준비, Action 실행	접촉 이벤트 분류 함수와 두 호출 제거
RunToCompletionEventProcessor	원본 이벤트와 불변 Context를 STM에 전달하고 Action 묶음을 순서대로 실행	기존 처리 구조 유지
TradingSTM	전역 전이 우선 평가, Region 1→C→B 평가와 결과 확정	기존 <code>handle()</code> 과 엔진 실행 순서 유지
global_transitions	G-02/G-03/G-07의 접촉 조건과 후속 동작 결정	실제 시장 관측 처리와 명시적 접촉 처리를 같은 구현에 통합
logic_registry	지원 REGIME, 시작 Guard, 전이 목록	미사용 상단 정책 enum·필드 제거

중요한 소스 위치는 **Controller의 관측 입력, Context 준비, 접촉 판단, 상단 동작**입니다.

2.2 Controller는 접촉 이름 대신 원본 관측을 전달합니다

`observe_market_evaluation()`은 유효한 실제 시장 평가를 `MARKET_DATA_UPDATED`로 생성합니다. 상단에 닿았는지, Case C가 회복했는지를 이 메서드에서 판단하지 않습니다.

아래는 현재 **실제 이벤트 생성 코드**의 발췌입니다. 이후 Python 블록도 독립 실행용 프로그램이 아니라, 링크된 소스의 일부입니다.

```
evaluation_time = self._clock()
event = TradingEvent(
    event_type=TradingEventType.MARKET_DATA_UPDATED,
    occurred_at=evaluation_time,
    priority=EventPriority.MARKET,
    event_id=f"market:{market_version}:{source_event_id}",
    lower_event_id=self._context.runtime.lower_event_id,
    candle_id=market.current_30m_candle_id,
    market_evaluation=market,
    market_version=market_version,
)
return self.enqueue_event(event) # Context mutation은 FIFO claim 직후 준비 단계에서만 수행한다.
```

이벤트를 큐에 넣는 시점에는 아직 그 시장값으로 Context를 덮어쓰지 않습니다. 뒤에 도착한 관측 때문에 앞선 이벤트의 판단 자료가 바뀌지 않게 하기 위해서입니다. 반환값도 주문 성공 결과가 아니라 큐에서 수락한 이벤트입니다.

큐에서 꺼낸 뒤 `_prepare_market_event_context()`는 다음을 수행합니다.

1. 원본 평가 자료형과 시장 version의 유효성을 검사합니다.
2. 이미 처리한 시장 version보다 앞으로 진행하는지 검사합니다. 큐 대기 때문에 현재 전체 시장 snapshot보다 오래된 관측일 수는 있습니다.
3. 해당 관측의 발생 시각까지 포지션·신호·C timer 경과시간을 계산합니다.
4. 현재 실행 상태의 `lower_event_id`를 연결하고 원본 시장값을 Context에 적용합니다.

여기서 새로 도착한 다른 가격을 가져오는 것은 아닙니다. 예를 들어 큐에 들어간 현재가가 2,378.06이었다면 그 이벤트는 그 가격으로 판단합니다. 다만 앞선 이벤트가 포지션을 열었다면 새 포지션 상태와 함께 판단합니다.

2.3 STM의 전역 전이가 접촉 조건과 Action을 함께 결정합니다

`TradingSTM._handle_locked()`는 기존대로 `handle_global_transition()`을 먼저 호출합니다. 이 함수가 `_handle_band_touch()`에 시장 접촉 판단을 맡깁니다. 새로운 분류기 클래스나 정책 프레임워크를 추가하지 않았습니다.

판단	실제 시장 관측에서 확인하는 핵심 조건	선택 결과
상단 접촉	양수 상단 밴드, 현재가 $\geq$ 상단, 현재 root가 <code>TRADE_MANAGEMENT</code>	아래 노출 검사에 따라 G-07 복귀 또는 Case 평가 계속
최초 하단 접촉	양수 하단 밴드, 현재가 $\leq$ 하단, <code>LOWER_TOUCH_WATCH</code> , lower scope 없음	G-02와 O-01/C-01/B-01
새 봉 하단 접촉	양수 하단 밴드, 현재가 $\leq$ 하단, 기존 scope 있음, 현재 봉 ID 있음·터치봉과 다름, <code>TRADE_MANAGEMENT</code> , 소유자·pending 주문 없음,	G-03과 O-01/C-01/B-01

판단	실제 시장 관측에서 확인하는 핵심 조건	선택 결과
	C 미소비 또는 회복 확인	
적용할 전역 접촉 전이 없음	동일 봉 반복, 접촉 아님, G-03 제한 등	TRADE_MANAGEMENT이면 기존 Region 평가 계속

최초·새 하단 접촉은 현재가로 판단합니다. 이전에 기록된 봉 저가만 낮았다는 이유로 새 하단 이벤트를 만들지 않습니다. G-03에 `pending_intent_id` 제한을 추가하는 등의 정책 변경도 이번에는 하지 않았습니다. 위 조건은 기존 분류와 Guard 가 실제 시장 경로에서 적용하던 조건을 옮긴 것입니다.

상단의 노출 판단은 다음 [실제 코드](#)에 있습니다.

```
runtime = context.runtime
# 거래소 주문 ID 발급 전의 준비·재시도 의도도 초기화하지 않는다.
if (runtime.pending_order_id is not None
    or runtime.pending_intent_id is not None
    or context.position.is_open):
    # 실제 시장 관측은 보유·주문 관리를 계속하고, 명시적 상단 event는 기존 no-op을 유지한다.
    if is_market_observation:
        return None
    return create_transition_outcome("G-07", state, exclusive=True)
```

여기서 `None`은 이 전역 함수가 지금 선택할 전이를 내놓지 않았다는 뜻입니다. 프로그램 종료도 아니고, `TradingSTM.handle()` 전체가 응답하지 않는다는 뜻도 아닙니다. STM은 다음 Region 판단을 진행하고 최종적으로 `TradingSTMResult`를 반환합니다.

또한 사용자가 말한 “아무것도 하지 않는 NaN”을 가격 데이터에 숫자 `NaN`을 넣는 방식으로 구현한 것이 아닙니다. 상단 접촉 자체로 매매 주문을 요청하지 않는 동작을 뜻합니다. 노출이 없을 때는 다음 하단 감시를 위해 내부 신호·timer를 정리하는 Action이 있습니다.

2.4 실제 관측, 명시적 접촉, 내부 재평가를 구분합니다

동일한 시장값을 Context에 가지고 있어도 모든 이벤트를 새 접촉으로 읽으면 안 됩니다.

입력	접촉 처리 의미
원본 시장 평가를 담은 <code>MARKET_DATA_UPDATED</code>	현재 시장 관측이므로 STM에서 상단·하단 정책을 평가합니다.
직접 전달한 <code>UPPER_BAND_TOUCHED</code> 등 명시적 이벤트	기존 호출 계약을 유지합니다. 상단에서 노출이 있으면 상태·Action 변경 없는 G-07을 반환합니다.
시장 평가를 담지 않은 내부 <code>MARKET_DATA_UPDATED</code> 또는 <code>timer.retry</code>	Context에 이전 가격이 남아 있어도 새로운 접촉으로 해석하지 않습니다. 해당 Case의 기존 검사 의미를 유지합니다.

실제 관측의 구분은 [다음 두 조건](#)입니다.

```
is_market_observation = (
    event_type is TradingEventType.MARKET_DATA_UPDATED
```

```
    and event.market_evaluation is not None
)
```

TradingEvent는 시장 평가와 market version이 함께 있어야 한다는 검증도 수행합니다. 공개 Controller 경로에서는 평가 자료형과 version 순서를 추가로 검사합니다. 명시적 이벤트의 기존 Guard를 바꾸지 않으면서도, 정상 시장 관측에는 밴드 유효성·봉 식별자 조건을 적용합니다.

2.5 정책 설정과 외부 계약은 어떻게 달라졌는가

UpperBandPolicy와 **TradingLogicConfiguration.upper_band_policy**를 제거했습니다. 이제 상단 동작 바꾸려면 실제 전역 전이 구현을 수정합니다. 레지스트리의 TYPE_0 지원, TYPE_1~4의 미지원 차단, 시작 Guard, 109개 전이 목록은 그대로입니다.

다음 계약도 유지했습니다.

- **TradingSTM.handle(event, context)**와 **TradingSTMResult**의 필드
- G-02/G-03/G-07 등 기존 전이 ID와 명시적 이벤트 이름
- UI/API와 저장 데이터 형식
- C 매수 우선권, Region 평가 순서, 기존 B/C 진입·보유·청산 기준
- 사용자 STOP의 취소·조화·청산·종료 경로

바뀐 관찰 결과는 실제 시장 입력의 이벤트 유형입니다. 하단이나 상단 가격이 들어와도 입력 로그에는 **MARKET_DATA_UPDATED**가 남습니다. “무엇이 들어왔는가”는 원본 이벤트 ID·시장 version·가격으로, “어떤 판단을 했는가”는 **transition_ids**로 확인합니다. 상단에서 포지션 관리를 계속한 경우 G-07이 없고 PB/PC 전이가 나오는 것이 정상입니다.

3. 수정으로 인한 장점

3.1 한 정책을 수정할 때 확인할 실행 위치가 줄었습니다

예를 들어 나중에 상단 접촉의 복귀 조건을 조정한다고 가정하겠습니다. 이전에는 Controller가 해당 입력을 어떤 이벤트로 보내는지와 STM이 그 이벤트에서 무엇을 하는지를 함께 수정·검토해야 했습니다. 지금은 **global_transitions**에서 조건과 결과를 함께 읽을 수 있습니다.

변경하려는 내용	수정 후 주된 확인 위치
상단에서 하단 감시로 복귀할 조건과 정리 Action	_handle_band_touch() 와 _handle_upper_band_touch() — 같은 전역 전이 파일
새 봉의 하단 이벤트를 열 수 있는 조건	같은 파일의 G-03 조건
시장 입력 version 검증·큐 보존 방식	Controller와 event processor
실제 주문 제출·조정 방식	Controller의 주문 실행 경로와 Gateway
B/C 매수 임계값 자체	기존 Case 전이와 공통 조건 정의

실행 위치가 줄어도 관련 테스트와 문서 검토는 필요합니다. “앞으로 무조건 파일 하나만 고치면 된다”는 뜻은 아닙니다. **접촉 정책**을 위해 **Controller**의 전략 분류와 미사용 레지스트리 값까지 동시에 맞출 필요가 없어졌다는 것이 실제 개선입니다.

3.2 전역 전이를 적용하지 않는 이유와 다음 평가가 연결됩니다

상단에서 포지션이 있으면 전역 함수가 결과를 반환하지 않고 STM이 기존 Region으로 진행합니다. 조건 판단과 다음 실행 경로가 같은 도메인 처리 흐름 안에 있습니다.

이 때문에 “상단에서 종료하지 않게 바꿨는데, 정작 보유 관리 이벤트가 전달되지 않는 상황”을 함께 확인하기 쉬워졌습니다. 전역 판단만의 단위 테스트와 Controller·큐·주문을 포함한 통합 테스트를 나누어 검증할 수도 있습니다.

3.3 테스트가 구현 위치보다 매매 결과를 확인하게 됩니다

기존 일부 테스트는 Controller의 비공개 분류 함수를 직접 호출해 이벤트 이름을 확인했습니다. 수정 후에는 시장 관측을 STM에 넣고 실제 전이·상태·Action을 검사합니다.

통합 테스트에서는 실제 공개 관측 경로를 사용해 다음을 검사합니다.

- 상단 뒤 세션과 지표가 유지되는가
- 다음 하단에서 새 scope가 열리고 B/C 감시가 재개되는가
- 상단 관측을 큐에 넣은 뒤 앞선 이벤트가 만든 주문·포지션도 보호하는가
- 주문 ID가 아직 없는 준비 재시도와 미결 SELL도 유지하는가
- 이전 하단 신호·timer가 새 하단에서 주문을 일으키지 않는가

따라서 함수의 위치를 바꾸는 것만으로 깨지는 검사보다, 잘못된 매매 동작을 발견하는 검사의 비중이 높아졌습니다.

3.4 개선 범위에는 한계도 있습니다

이번 변경은 Controller 전체를 작게 나누는 작업이 아닙니다. Controller에는 계좌·주문·저장·재시도·복구 책임이 여전히 많이 있습니다. 특히 `_preparation_signal_valid()`처럼 주문 실행 경계에서 현재 신호를 다시 확인하는 코드도 남아 있습니다.

또한 다음 내용을 이번 성과로 주장할 수는 없습니다.

- 거래 처리 속도가 빨라졌다는 성능 개선: 별도 성능 측정을 하지 않았습니다.
- 모든 전락 조건이 한 파일에 모였다는 설명: Case별 조건과 실행 안전 검사는 각 책임 위치에 남아 있습니다.
- 실거래의 모든 상황에서 오류가 없다는 보장: 테스트는 정해진 입력과 가짜 거래소로 검증했습니다.

이번에 확인한 장점은 기존 매매 결과를 유지하면서 시장 접촉 정책의 책임과 검사 위치를 명확히 한 것입니다.

4. 수정 후 코드를 흐름대로 따라가는 예시

4.1 먼저 읽을 코드의 순서

순서	실제 함수	확인할 내용
1	<code>observe_market_evaluation</code>	검증 후 원본 시장 관측 생성·큐 등록
2	<code>process_next</code>	이벤트 꺼내기, Context 준비 요청, 같은 snapshot으로 STM 호출
3	<code>_prepare_market_event_context</code>	원본 시장값과 최신 실행 상태 연결
4	<code>_handle_locked</code>	전역 전이 먼저, 없으면 Region 1→C→B
5	<code>_handle_band_touch</code>	G-02/G-03/G-07 선택 또는 Case 평가 계속

순서	실제 함수	확인할 내용
6	<code>_commit</code>	STM 상태와 순서 있는 Action 결과 확정
7	<code>_execute_action</code>	Context.scheduler 주문 효과 실행
8	<code>_record_indicator_evaluation</code>	판단 입력,결과와 표시용 지표 저장

표의 2번 함수가 3번 함수를 callback으로 호출한 뒤 4번으로 진행합니다. 파일이 나뉘어 있어도 이 호출 순서로 읽으면 한 시장 관측의 처리를 따라갈 수 있습니다.

4.2 예시 A: 하단 접촉 → 상단 복귀 → 같은 봉의 하단 재접촉

시작 상태는 자동매매 세션이 실행 중이고, 포지션·pending 주문·pending intent가 없는 **LOWER_TOUCH_WATCH**입니다. 아래 수치는 기존 장에 재현 테스트에서 사용하는 값의 핵심 부분을 줄여 표시했습니다. 후속 재접촉은 연결 동작을 검증하기 위한 합성 입력이며, 실제 시세 전체를 재생한 백테스트는 아닙니다.

단계	현재가	실시간 밴드와 보조값	기대 동작
첫 하단	2,378.06	하단 약 2,379.255257, 상단 약 2,417.962743, BBW 약 0.016137, %B 약 -0.030879	새 하단 이벤트 생성, B/C 진입 감시
이후 상단	2,419.61	상단 약 2,419.557349, %B 약 1.001402	이전 하단 상태 정리 후 하단 대기
상단 위 반 복	같은 상단 관측 반복	주문·포지션 없음	하단 대기 유지
같은 봉 하 단 재접촉	첫 하단 관측값 재사용	최초 터치와 같은 30분봉 ID	새 하단 이벤트로 B/C 감시 재개

㉠ 첫 하단 관측을 큐에 넣습니다.

`observe_market_evaluation()`은 현재가가 하단보다 낮더라도 **LOWER_BAND_TOUCHED**를 만들지 않습니다. **MARKET_DATA_UPDATED**와 원본 평가, 발생 시각, source ID, market version을 보존합니다. 이 시점에는 아직 상태가 **LOWER_TOUCH_WATCH**입니다.

㉡ 처리 시점의 **Context**를 준비합니다.

`process_next()`가 이벤트를 꺼내 `_prepare_market_event_context()`를 호출합니다. 아직 포지션이나 이전 신호가 없으므로 관련 경과시간은 0입니다. 같은 원본 시장 평가를 Context에 적용한 다음 불변 snapshot을 STM에 전달합니다.

㉢ STM이 **G-02**를 선택합니다.

전역 접촉 함수는 상단 조건이 아니고, 양수 하단 밴드에 현재가가 닿았으며, root가 하단 감시이고 scope가 없다는 것을 확인합니다. 다음 결과를 만듭니다.

```
transition_ids = (G-02, O-01, C-01, B-01)
root_state = TRADE_MANAGEMENT
ownership_state = NO_POSITION
```

```
case_b_signal_state = B_WAIT_TOUCH
case_c_signal_state = C_WAIT_SETUP
```

이 네 ID는 네 번의 이벤트 처리를 뜻하지 않습니다. 한 **G-02** 처리에서 복합 상태와 세 **Region**의 최초 상태를 함께 연 결과 임니다.

Action의 순서는 **OpenLowerEvent** → 하단 이벤트 초기화 patch → **position_owner=None** patch → **QueueEvent(ACTIVATE_TRADE_MANAGEMENT)** 입니다. 하단 이벤트 초기화 helper는 이 접촉의 BBW로 **case_b_enabled**를 정하고, C 활성화·소비·복구 등 새 scope의 flag를 초기화합니다.

㉠ Controller가 Action을 적용하고 후속 이벤트를 등록합니다.

processor는 판단 당시 Context version이 유지됐는지 확인하고, **_execute_action()**을 통해 실제 Context를 변경합니다. **OpenLowerEvent**는 하단 ID, 접촉 시각, 터치봉, 밴드와 BBW를 고정합니다. **QueueEvent**의 실제 큐 삽입은 Action 묶음이 끝난 다음입니다.

따라서 다음 microstep의 **ACTIVATE_TRADE_MANAGEMENT**는 새 lower scope와 초기화된 Context를 읽습니다.

㉡ C와 B가 각각 감시 상태를 확정합니다.

이 예시의 %B는 약 -0.030879여서 C setup의 $%B \leq -0.15$ 조건을 충족하지 않습니다. C-02는 **C_WAIT_SETUP**을 유지하고 다음 재평가를 예약합니다. B는 접촉 BBW가 0.02 미만이므로 B-03으로 **B_WAIT_SIGNAL**에 들어갑니다.

```
첫 microstep: G-02 + 0-01 + C-01 + B-01
다음 microstep: C-02 + B-03
```

```
최종: TRADE_MANAGEMENT / NO_POSITION / B_WAIT_SIGNAL / C_WAIT_SETUP
```

여기까지는 매수 조건을 감시하기 시작한 것이므로 주문 제출은 없습니다. B 또는 C의 실제 진입 조건이 나중에 맞아야 매수 요청을 만듭니다.

㉢ 이후 상단 관측은 같은 입력 형식으로 들어옵니다.

현재가 2,419.61과 상단 약 2,419.557349를 가진 **MARKET_DATA_UPDATED**가 큐와 Context 준비를 거쳐 STM에 도달합니다. **_handle_band_touch()**가 상단 접촉을 확인하고 **_handle_upper_band_touch()**를 호출합니다.

실제 포지션, pending 주문 ID, pending intent가 모두 없으므로 G-07이 하단 감시 복귀를 선택합니다. STM의 다음 root는 **LOWER_TOUCH_WATCH**이고 하위 Region 필드는 비활성 상태가 됩니다.

반환된 Action과 실제 변경은 다음 순서입니다.

순 서	Action	실제로 정리되는 내용
1	CloseLowerEvent	하단 이벤트 ID, 터치 시각·봉, 터치 가격 정보 ·BBW
2	ResetCaseBContext	이전 B 신호 생성 여부·봉·시각과 B 제어 flag

순 서	Action	실제로 정리되는 내용
3	<code>ResetCaseCContext</code>	이전 setup 봉, flush 저점·%B·시각, 회복 timer, 진입선, trailing 자료와 C flag
4	pending/owner/phase patch	관련 진행 필드를 비우고 <code>trading_phase=IDLE</code> 적용
5	<code>CancelScheduledEvaluation(scope="lower-event")</code>	하단 이벤트 ID를 가진 예약 평가 제거; 세션 전역 예약은 보존

Context 필드 초기화는 `TradingContext`의 `close/reset` 메서드, 예약 취소는 `_EventDrivenScheduler.cancel`에 있습니다.

이것이 “이전 신호·timer를 정리한다”는 말의 구체적인 의미입니다. 이전 하단에서 만들어진 매수 근거를 새로운 하단 이벤트에 이월하지 않도록 메모리의 전략 진행 자료와 예약을 정리합니다. 거래 이력 파일이나 실제 보유 자산을 삭제한다는 뜻이 아닙니다.

결과는 다음과 같습니다.

```

세션 status          = RUNNING
STM root_state       = LOWER_TOUCH_WATCH
runtime.trading_phase = IDLE
runtime.lower_event_id = None
실제 신규 주문       = 없음
StopTradingRuntime 요청 = 없음

```

지표 저장소는 성공한 처리 결과를 받아 새 상태에 맞는 하단 가격 조건을 제공합니다. “B/C의 이전 행이 사라졌다”와 “실행 중인 전략이 없어서 아무 지표도 없다”는 다른 상태입니다. 여기서는 실행 세션이 유지되고 하단 접촉 조건을 감시합니다.

㉠ 같은 봉에서 다시 하단에 닿으면 G-02부터 새로 시작합니다.

G-07이 이전 lower scope를 닫았으므로 같은 30분봉이라는 이유만으로 재접촉이 차단되지 않습니다. 현재 root가 `LOWER_TOUCH_WATCH`이고 scope가 없어서 G-02가 새 ID와 새 터치 snapshot을 만듭니다. C-02/B-03을 거쳐 B/C 감시가 재개됩니다.

다만 B/C가 항상 둘 다 활성화된다고 일반화하면 안 됩니다. 새로운 접촉의 BBW가 정확히 0.02이면 B는 B-02로 종료되고 C 감시만 남습니다. 재진입은 기존 조건을 다시 평가하는 과정입니다.

4.3 예시 B: B 포지션을 보유한 채 상단에 닿습니다

시작 상태는 `CASE_B_HOLDING`이고 실제 포지션 수량이 양수입니다. 테스트는 상단보다 높은 가격과 함께 B의 강한 반등 조건인 $\%B \geq 0.60$ 유지, $\text{slope} > 0.08$ 유지 값을 전달합니다.

1. Controller는 이번에도 `MARKET_DATA_UPDATED`를 만듭니다.
2. 전역 접촉 함수는 상단을 확인하지만 `context.position.is_open`이 참입니다.
3. 실제 시장 관측이므로 `_handle_upper_band_touch()`는 `None`을 반환합니다. G-07 초기화 Action은 생기지 않습니다.

4. `_handle_locked()`는 Region 1 평가를 계속합니다. `_resolve_region_one_event()`가 B 보유 상태에 맞게 `RETRY_CASE_B_CONDITION_CHECK`로 해석합니다.
5. B 보유 전이 PB-05가 강한 반등 조건을 선택하고 `CASE_B_UPPER_TREND`를 후속 이벤트로 요청합니다.
6. 다음 microstep의 PB-14가 `CASE_B_TREND_HOLD`로 전환합니다. 이어 PB-15가 추세 유지 조건을 확인합니다.

실제 회귀 테스트에서 관측한 순서는 다음과 같습니다.

MARKET_DATA_UPDATED

- G-07 적용 없음
- PB-05
- 다음 이벤트 PB-14
- 다음 이벤트 PB-15

최종: `CASE_B_TREND_HOLD`, 기존 보유 수량 유지, 세션 `RUNNING`

상단 접촉 자체가 추가 주문을 만든 것은 아닙니다. 같은 관측에 포함된 기존 B 조건을 정상 평가한 것입니다. C 보유 테스트에서는 같은 원리로 `PC-05` → `PC-10` → `PC-11`을 거쳐 `CASE_C_TP_TRAILING`으로 진행합니다. 이 경로가 보존돼야 "상단에서도 기존 전략을 계속 관리한다"는 요구를 만족합니다.

직접 `UPPER_BAND_TOUCHED`를 주입하는 호환 경로는 다릅니다. 노출이 있는 경우 `consumed=True`, `transition_ids=(G-07,)`, 상태 동일, 빈 Action 결과를 반환합니다. 이 직접 이벤트는 일반 가격 관측을 대신하는 수단으로 사용하지 않습니다.

4.4 예시 C: 큐에 넣을 때는 없던 주문이 처리 전에 생깁니다

다음 입력 두 개를 처리하기 전에 큐에 넣었다고 하겠습니다.

큐 적재 시점: 포지션·pending 주문 없음

관측 A: C 회복 매수 조건 충족

관측 B: 상단 가격

먼저 A가 처리되어 C-12가 매수 의도를 예약하고 주문을 요청합니다. 그 결과가 미결이면 `pending_order_id` 또는 `pending_intent_id`가 남고, 즉시 체결이면 실제 Position과 체결 피드백이 반영됩니다. 내부 주문 결과는 대기 중 시장 관측보다 먼저 처리합니다.

이후 B가 처리될 때에는 B 자신의 원본 상단 가격을 읽되, 최신 주문·포지션 상태를 포함한 Context로 판단합니다. 따라서 관측 B를 적재할 때 무포지션이었다는 이유로 G-07 초기화를 실행하지 않습니다. 미결 주문이면 그대로 보호하고, 체결 포지션이면 해당 Case 관리를 계속합니다.

이 사례는 **주문 생성 전 적재한 상단 관측 테스트**와 **체결 전 적재한 상단 관측 테스트**에서 확인합니다. 두 테스트는 주문·체결을 직접 가정해 결론만 검사하는 것이 아니라 Controller·STM·큐와 가짜 거래소의 실제 처리 경로를 실행합니다.

4.5 예시 D: 상단과 하단을 미리 적재했을 때의 scope와 timer

이미 하단 이벤트 L1이 열려 있는 상태에서 상단 관측 U와 다음 하단 관측 L을 함께 적재하면, 둘 모두 적재 당시에는 L1을 가지고 있을 수 있습니다.

처리 순서	처리 직전 scope	수행 결과
U 처리	L1	G-07이 L1 종료, 하단 감시 복귀
L 처리	없음	Controller가 처리 시점 scope로 연결하고 STM이 G-02로 L2 생성
L1의 지연 timer 도착	현재 L2	timer의 원래 ID는 L1로 유지되므로 불일치 검사에서 배제

새 시장 관측을 현재 scope에 연결하는 것과 이전 timer를 새 scope로 바꾸는 것은 다릅니다. 후자를 허용하면 이전 신호의 예약이 새로운 하단 이벤트에서 동작할 수 있습니다. 현재 `_prepare_market_event_context()`는 원본 시장 평가가 없는 timer의 scope를 바꾸지 않습니다.

기존 `_is_stale_lower_event()`는 양쪽 ID가 모두 있고 서로 다를 때 배제합니다. 모든 과거 이벤트를 시간만으로 무조건 버리는 필터라고 설명하면 실제 코드와 다릅니다.

4.6 예시 E: 아직 거래소 주문 ID가 없는 준비 재시도

`SubmitOrder`를 수행하다 일시적인 주문 준비 오류가 발생할 수 있습니다. 테스트에서는 Gateway의 준비 단계에 `TimeoutError`를 주입합니다. 이때 거래소에 주문을 제출하지 않았더라도 다음 값이 존재할 수 있습니다.

```
pending_order_id = None
pending_intent_id = 기존 매수 의도 ID
예약              = CASE_B_BUY_RETRY
```

이후 상단 관측이 들어오면 G-07은 pending intent를 보고 하단 초기화를 하지 않습니다. 기존 매수 의도와 BUY 재시도 예약이 유지됩니다. 다만 다른 Case의 정상적인 감시 재평가가 자신의 예약을 갱신하는 것은 가능합니다. “상단에서 모든 내부 변화가 정지한다”는 뜻은 아닙니다.

나중에 준비 재시도 시점이 되면 기존 `_preparation_signal_valid()`가 현재 신호의 유효성을 다시 확인합니다. 신호가 만료됐다면 `ORDER_PREPARATION_EXPIRED`로 감시 상태에 복귀할 수 있습니다. **상단 접촉이 의도를 지우는 것과 원래 주문 실행 절차가 만료된 신호를 폐기하는 것은 서로 다른 판단입니다.** 이 주문 준비 유효성 검사는 이번 리팩터링 범위에 포함하지 않았습니다.

4.7 코드를 읽으며 결과를 확인하는 방법

실제 흐름을 점검할 때는 다음 순서로 기록을 확인하면 됩니다.

1. 원본 이벤트 ID·market version·발생 시각·현재가를 확인합니다. 접촉이 있어도 입력 유형은 `MARKET_DATA_UPDATED`입니다.
2. 판단에 사용된 `state_before`와 주문·포지션·scope 상태를 확인합니다.
3. `transition_ids`를 읽습니다. 하단 신규 생성은 G-02, 새 봉 재초기화는 G-03, 노출 없는 상단 복귀는 G-07입니다.
4. `state_after`와 순서 있는 `action_requests`를 확인합니다. `consumed=True`만으로 주문이 발생했다고 해석하지 않습니다.
5. 주문 요청이 있다면 Controller의 주문 실행·체결·저장 결과까지 확인합니다.
6. 세션 상태와 활성 전략·지표를 확인합니다. root가 하단 감시로 바뀌어도 세션은 실행 중일 수 있습니다.

부록. 검증 근거와 이번 문서의 범위

2026-09-17 구현 검증에서는 변경 전 기존 상단 연속성 시나리오를 기록하고 최종 코드와 비교했습니다. 7개 시나리오·35개 관측 지점에서 상태, 전이, Action 순서·인자, Context, 예약 평가, 주문 조회, 제출 주문, UI snapshot이 동일했습니다. 세션 UUID를 고정하고 직렬화했으며, 입력 이벤트 유형 변경은 별도 관측·로그 테스트로 확인했습니다.

검증 항목	결과 및 소스
변경 전후 동등성	비교 단계 목록과 해시 , 35개 지점 동일
Controller 없이 접촉 정책 판단·명시적 입력 호환	test_market_observation_policy.py
상단→하단 재진입, BUY/SELL/준비 의도 보호	test_upper_band_continuation.py
큐 대기 중 체결·시간 경계	test_post_buy_indicators.py
원본 관측과 로그 유형	test_trading_session_flow.py , test_runtime_logging_flow.py
전체 회귀	백엔드 1,233개 중 1,223개 통과·실제 거래소 opt-in 10개 제외, 관련 UI 54개·타입 검사 통과

전체 결과와 실행 환경은 [변경·검증 보고서](#)에 있습니다. 이 결과는 동일 입력에서 기존 동작이 유지됐다는 근거이며, 모든 시장 조건이나 실거래 운영을 완전 탐색했다는 의미는 아닙니다. 기존 구조 설명 문서는 상단 정책·시장 전달·주문 준비 경로·소스 색인을 현재 코드에 맞췄고, 이번 문서 작성 과정에서 매매 코드는 추가 변경하지 않았습니다.